

What Message Passing Actually Does

One primitive, four layer types, and the ceiling it cannot pass

src/layers.py · src/graph_conv.py · 2026-08-26

1 The one-sentence answer

Message passing is a **fixed number of synchronous rounds of local information exchange**. In each round, every node simultaneously reads the current state of its direct neighbours, compresses what it read into a single vector, and rewrites its own state from that summary plus its old state.

Everything a message-passing GNN can compute follows from that one sentence — including, importantly, everything it **cannot** compute.

A node's representation after L rounds is a function of its **L -hop neighbourhood** — the subgraph of everything reachable within L edges — and of nothing else.

That single invariant is the lever the whole thesis pushes on. Depth is not a capacity knob here; depth is reach.

2 The primitive: one round in three steps

Write $h_i^{(\ell)}$ for the state of node i after round ℓ , and $\mathcal{N}(i)$ for its neighbours. One round is:

$$\begin{aligned} (1) \text{ message } & m_{j \rightarrow i} = \text{MSG}\left(h_i^{(\ell)}, h_j^{(\ell)}, e_{ji}\right) \\ (2) \text{ aggregate } & a_i = \bigoplus_{j \in \mathcal{N}(i)} m_{j \rightarrow i} \\ (3) \text{ update } & h_i^{(\ell+1)} = \text{UPD}\left(h_i^{(\ell)}, a_i\right) \end{aligned}$$

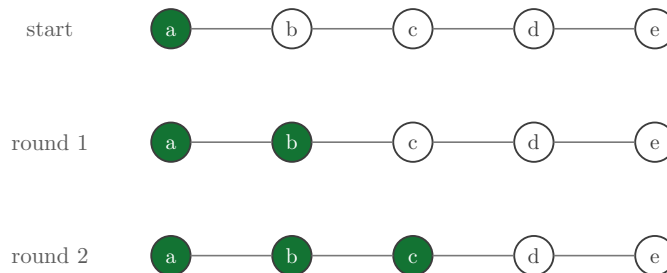
Three design choices, and they are the **only** three: what a message contains, how a multiset of messages collapses to one vector, and how a node folds that summary back into itself.

Two properties are forced by the shape of the formula, not by any particular choice:

- **Permutation invariance.** $\oplus$ runs over a set, so relabelling the graph cannot change the answer. This is the reason GNNs work on graphs at all, and it is bought at the cost of item (2) below.
- **Information loss at the aggregator.** A whole neighbourhood — however large — is squeezed into one fixed-width vector. Nothing downstream can recover what $\oplus$ threw away.

2.1 What “one round” buys you, concretely

Take reachability. Give every node a one-hot marker of itself, use $\oplus = \vee$ (boolean OR), and $\text{UPD} = \vee$. Then $h_i^{(\ell)}$ is exactly the set of nodes within ℓ hops of i . Round by round:



The lit frontier advances **exactly one edge per round**. To learn that **a** reaches **e** on this path you need four rounds; no width, no learning rate, and no amount of data substitutes for the fourth round. This is the entire content of the depth↔diameter story that the connectedness experiments keep running into.

3 The four layers in this repo are one template

`src/layers.py` implements four message-passing layers. They look different and they are not — each one is the template above with different fills for MSG / $\oplus$ / UPD.

Layer	Message	Aggregate	Update
GCNConv	$c_{ij} \cdot Wh_j$, with $c_{ij} = (d_i d_j)^{-1/2}$	sum	none — self-loop folds i in
SAGEConv	h_j (raw)	mean	$W \cdot [h_i \mid a_i]$
GATConv	$\alpha_{ij} \cdot Wh_j$, α learned per pair	sum (per head)	none — self-loop folds i in
GINConv	h_j (raw)	sum	$\text{MLP}((1 + \varepsilon)h_i + a_i)$

Reading the table as a progression is the useful part:

- **GCN** fixes the neighbour weights from degree alone. Cheap, and blind to what the neighbour actually contains.
- **SAGE** keeps self and neighbourhood in **separate slots** of the concatenation, so the update can treat “what I am” and “what surrounds me” differently. GCN cannot: the self-loop dissolves the two into one sum.
- **GAT** makes the weights content-dependent — α_{ij} is computed from h_i and h_j — so a node can decide, per round, which neighbours matter.
- **GIN** uses sum with an MLP update, which is the choice that maximises what survives step (2). See below.

3.1 Why the aggregator choice is the expressiveness choice

Mean and max are **lossy in a specific way**: mean cannot tell $\{a, a\}$ from $\{a\}$, and max cannot tell $\{a, b\}$ from $\{a, b, b\}$. Sum distinguishes both. Xu et al. (2019) turn this into the sharp statement: with a sum aggregator and an injective update, a message-passing GNN is **exactly as discriminative as the 1-dimensional Weisfeiler–Lehman colour refinement test** — and no message-passing scheme of this form can beat 1-WL.

That is a ceiling, not a floor. It bites in this repo directly: two graphs that 1-WL cannot separate are indistinguishable to **GINConv** at any depth and any width. The isomorphism experiments (`analyze_iso.py`, the `iso_wl` configs) live precisely at that boundary, which is why they need something other than a message-passing prior to move.

4 What message passing cannot do

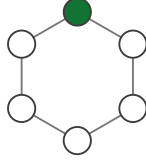
Three distinct failure modes, often conflated. They are separate and have separate fixes.

Failure	Mechanism	Fix that works
under-reaching	L rounds see L hops; anything farther than the diameter is invisible, period	more depth, or non-local layers
over-squashing	an exponentially growing receptive field is forced through one fixed-width vector at a bottleneck edge	rewiring, virtual nodes, global attention
over-smoothing	repeated averaging is a diffusion; node states converge to a constant as L grows	residuals, normalisation

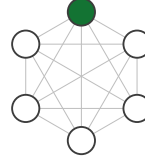
Note the trap: the fix for under-reaching (add layers) **feeds** over-smoothing, and the deeper stack squeezes more of the graph through the same width, feeding over-squashing. This is why “just make it deeper” stops working well before the diameter on real graphs — and why `GraphConvNet` applies a residual and an optional norm around every layer (`src/graph_conv.py:157`), which buys depth without the diffusion collapse.

5 Why this repo contrasts it with global attention

`GlobalAttnConv` (`src/layers.py:135`) drops into the **same** pipeline slot as the four layers above — same `forward(x, edge_index, batch)` signature, same residual/norm/ReLU wrapper — and changes exactly one thing: it removes the restriction that j must be a neighbour of i . Every node attends to every node in its graph.



message passing: reach +1 hop



global attention: reach $\times 2$

The consequence is a change in the **depth budget**, not in the per-layer cost of reasoning. Message passing extends the reachability horizon **additively** — one hop per round — so it needs $\Theta(\text{diameter})$ layers. All-pairs attention can implement the doubling trick, squaring the reachable set each layer, so it needs $O(\log N)$ (Sanford et al. 2024). On a graph of diameter 16 that is 16 layers versus 4.

This is the same distinction as PRAM/MPC round complexity, and it is the reason the experiments in this repo hold the task fixed and vary only the layer type: **the question is never “can the architecture represent the answer”, it is “how many rounds of communication does the answer need, and does the stack have that many”**.

The `local=True` flag on `GlobalAttnConv` is the controlled middle: attention machinery, but masked to the adjacency, so reach again grows one hop per layer. It isolates “is it the attention or is it the reach?” — the answer being reach.

6 Where it lives in the code

<code>src/layers.py:8</code>	<code>GCNConv</code> — degree-normalised sum
<code>src/layers.py:36</code>	<code>SAGEConv</code> — mean, self kept separate
<code>src/layers.py:58</code>	<code>GATConv</code> — learned per-edge attention, multi-head
<code>src/layers.py:108</code>	<code>GINConv</code> — sum + MLP, the 1-WL-tight one
<code>src/layers.py:135</code>	<code>GlobalAttnConv</code> — all-pairs; the non-message-passing contrast
<code>src/graph_conv.py:72</code>	<code>GraphConvNet</code> — the shared embed $\rightarrow L \times \text{layer} \rightarrow \text{readout engine}$
<code>src/gnn.py:5</code>	<code>GNN</code> — <code>GraphConvNet</code> restricted to local aggregators

Steps (1) and (2) of the primitive are not written out by hand: all four layers subclass PyG’s `MessagePassing` and get them from `propagate()`, declaring $\oplus$ via `aggr=` in the constructor and `MSG` via the `message()` method. The `forward()` body is step (3), the update. Reading the four `message()` methods side by side is the fastest way to see that the differences between these architectures are genuinely small.

7 Summary

1. Message passing is L synchronous rounds of **message** $\rightarrow$ **aggregate** $\rightarrow$ **update** over the edge set.
2. After L rounds a node knows its L -hop neighbourhood and nothing more. Depth is reach.

3. The aggregator is where information is destroyed; sum destroys the least, which is why GIN is 1-WL-tight — and 1-WL is the ceiling for the whole family.
4. Its three failure modes — under-reaching, over-squashing, over-smoothing — are distinct, and the fix for the first aggravates the other two.
5. Global attention is the same pipeline with the neighbour restriction lifted: $O(\log N)$ rounds instead of $\Theta(\text{diameter})$.

Kipf & Welling 2017 (GCN) · Hamilton et al. 2017 (GraphSAGE) · Veličković et al. 2018 (GAT) · Xu et al. 2019 (GIN / 1-WL) · Alon & Yahav 2021 (over-squashing) · Sanford et al. 2024 (depth $\leftrightarrow$ MPC rounds)